

Differentiable Finite Elements in MFEM

MFEM Workshop 2026, Atlanta

Leonardo Molinari
Julian Andrej

Center for Applied Scientific Computing | LLNL



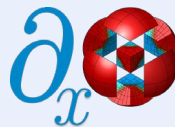
LLNL-CFPRES-2024604

This work was performed under the auspices of the U.S. Department of Energy by Lawrence Livermore National Laboratory under contract DE-AC52-07NA27344. Lawrence Livermore National Security, LLC

What you'll leave with

1

What ∂ FEM is



Automatic differentiation **constrained to the quadrature point**. You write the physics there; MFEM derives the action, the Jacobian, its transpose, assembled.

2

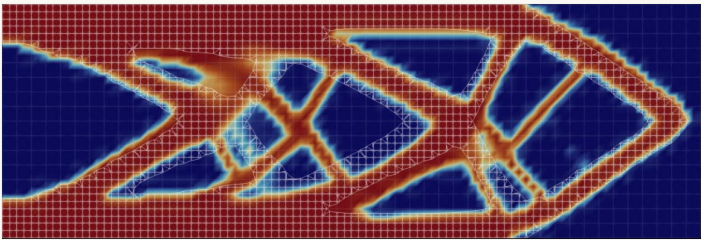
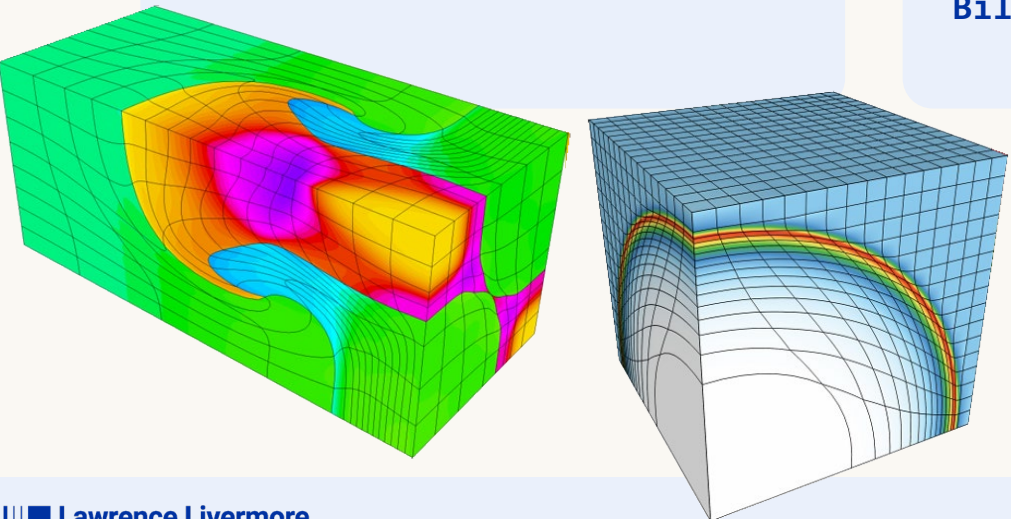
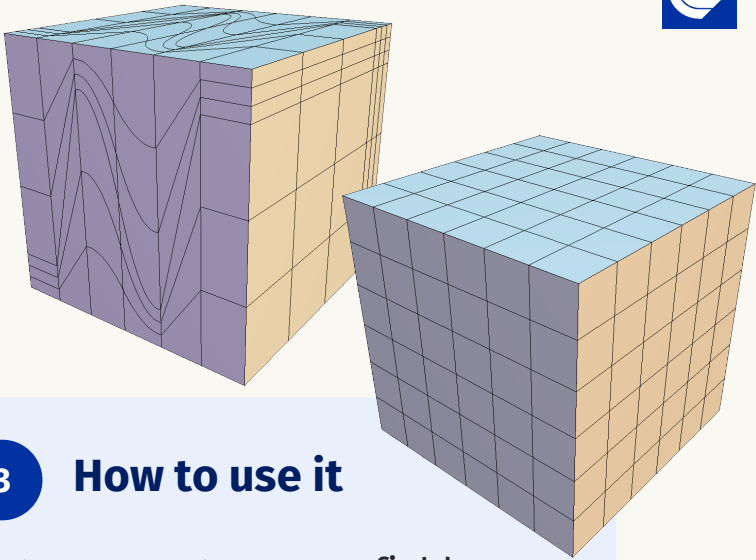
When to reach for it

Sensitivities, PDE-constrained optimisation, inverse problems, topology optimization, or any custom physics you would otherwise write a **BilinearFormIntegrator** for.

3

How to use it

A **FieldDescriptor** per field, **FieldOperators** saying how each is evaluated, and a **q-function** carrying the physics.



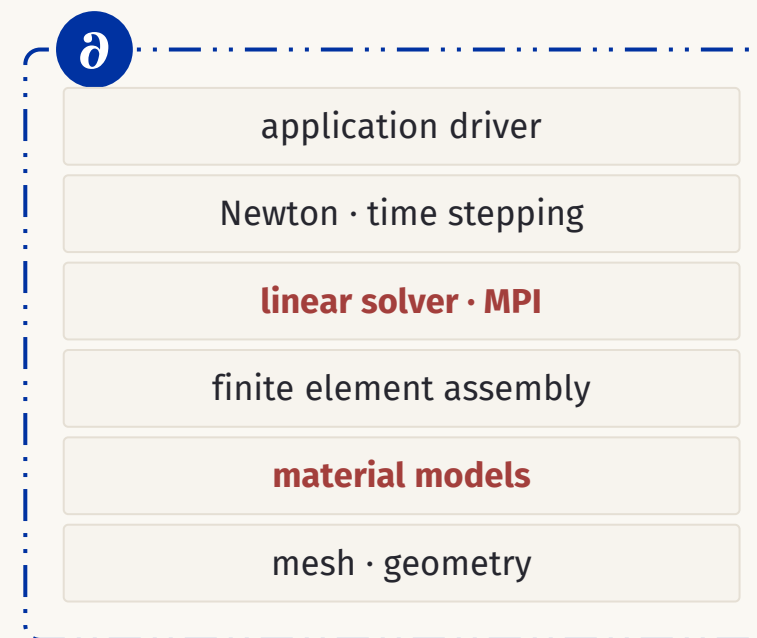


The main idea

Every MFEM operator already factors into a **topological part** and a **pointwise part** (physics).
 ∂ FEM differentiates only the second one.

Why not just wrap the whole program?

- Applied at the top level, AD has to see through **everything underneath**:
 - complicated program structure
 - non-trivial object types
 - communication layers such as **MPI**
- Production codes compose **physics, languages, models** and **libraries**
- The overhead is infeasible, and worsen on GPUs.

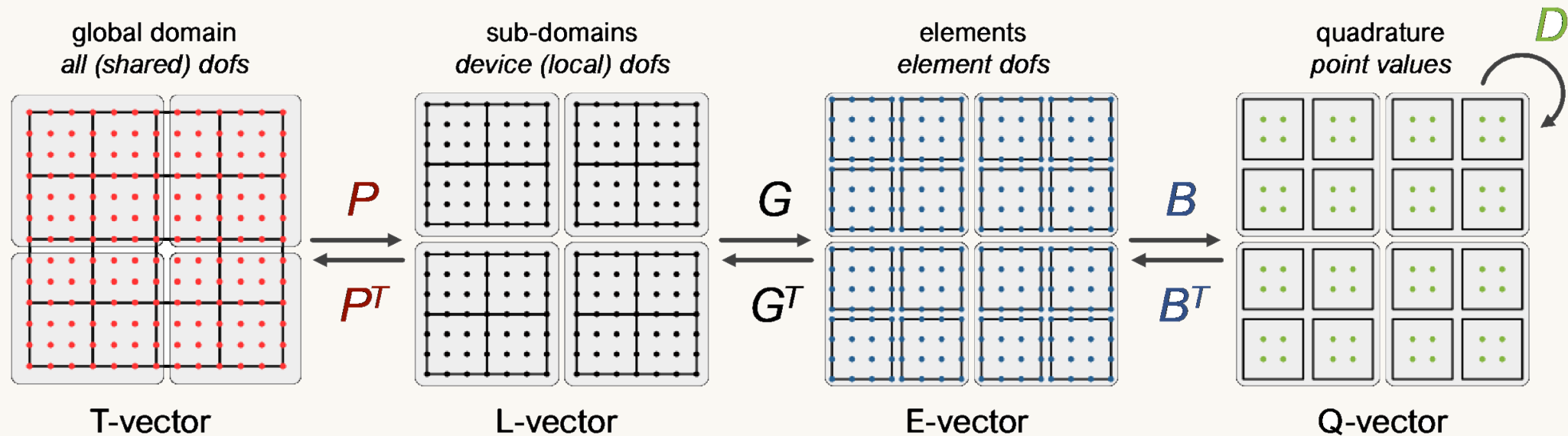


One AD call at the top, and it has to see through **all of it**.

∂FEM instead constrains AD to the **quadrature point level!**

Every MFEM user knows how an operator is written:

$$A_p(u) = P^T G^T B^T D(B G P u)$$



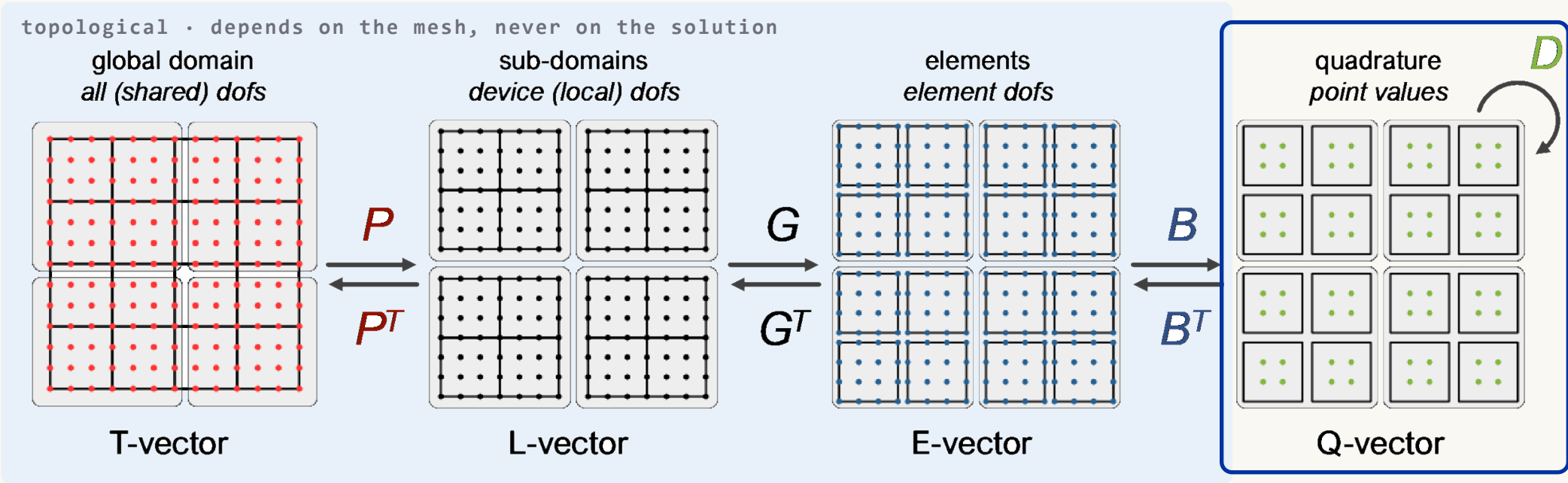
P, G and B move data around. Only **D** knows any physics.

Only D is differentiated

$$J_p(u) = P^T G^T B^T J_D(u_q) B G P$$

$$u_q = BGPu$$

$$J_D = dD(u_q)/du_q$$



The same decomposition, wrapped around the Jacobian of **D** alone.

AD engine: Enzyme, or dual numbers

Two AD tools, selected when MFEM is configured. They differ only in the scalar type your differentiated arguments carry, so one `#ifdef` covers both.

```
#ifdef MFEM_USE_ENZYME
using dscalar_t = real_t;           // Enzyme differentiates plain code
#else
using dscalar_t = dual<real_t, real_t>; // dual number fallback
#endif
```



Enzyme AD

Enzyme (recommended)

A **compiler plugin** that differentiates at the **LLVM IR level**, after optimization, with forward and reverse mode support, including on GPU kernels. Working at the IR level lets it differentiate across multiple languages (C, C++, Julia, Rust, Fortran)

dual<real_t, real_t> (fallback)

Header-only forward mode by operator overloading. No external dependencies and it debugs easily.

Building a ∂ FEM operator

∂ FEM is centered around the **DifferentiableOperator** class.

Like a BilinearForm, you **construct it on a mesh**, **add integrators** to it, and **apply** it. Except, for a you write the integrand instead of picking one from a catalogue.

Three pieces

FieldDescriptor

which space a field lives in

An id you pick, paired with a **ParFiniteElementSpace**, a **VectorQuadratureSpace**, or a **ParameterSpace**.

FieldOperator

how it is evaluated at a q-point

Inputs<...> fixes what the q-function gets, and in which order.
Outputs<...> fixes the test basis.

the q-function

the physics at one q-point

Functor describing the physics at the quadrature point level
(the only part really **differentiated**)

```
struct NonlinearDiffusionQF { ... }; // the q-function

std::vector<FieldDescriptor> inputs = {{U, &fes}, {Coords, mesh_nodes_fes}};
std::vector<FieldDescriptor> outputs = {{U, &fes}};

DifferentiableOperator dop(inputs, outputs, pmesh);

NonlinearDiffusionQF qf;

dop.AddDomainIntegrator<LocalQFBackend>(
    qf,
    Inputs<Value<U>, Gradient<U>, Gradient<Coords>, Weight> {},
    Outputs<Gradient<U>> {},
    ir, domain_attr, Derivatives<U> {});
```

Field operators

The **B** of the decomposition: how each field is evaluated at the quadrature points.

Field operator	At a quadrature point	Q-function argument
Value<ID>	the interpolated value	scalar_t , tensor<scalar_t, V>
Gradient<ID>	the gradient, in reference coordinates	tensor<scalar_t, D> , tensor<scalar_t, V, D>
Hessian<ID>	the hessian, in reference coordinates	tensor<scalar_t, D, D> , tensor<scalar_t, V, D, D>
Identity<ID>	the data as is, quadrature/parameter spaces	scalar_t , tensor<scalar_t, ...>
Weight	the integration rule weight (no field id)	real_t
FunctionalValue<ID>	output only: sum into a scalar functional	scalar_t

- In **Inputs** the position fixes the argument; in **Outputs** the operator picks the test basis.
e.g., **Value<U>** mass-like, **Gradient<U>** diffusion-like.
- D is the mesh dimension, V the space's vector dimension, **scalar_t** is **real_t** under Enzyme, or **dual_t** for the fallback.

From the weak form to the q-function

THE WEAK FORM

$$r(u; v) = \int_{\Omega} \kappa(u) \nabla u \cdot \nabla v \, dx = 0$$

for all $v \in V_h$, with $\kappa(u) = 1 + u^2$

AT THE QUADRATURE POINTS

$$\sum_q \kappa(u_q) \nabla_x u_q \cdot \nabla_x v_q \det(J_q) w_q$$

THE DECOMPOSITION

$$A_p(u) = P^T G^T B^T D(BGPu)$$

Only **D** is written, and only **D** is differentiated. The test-function side is the transpose you already have.

THE Q-FUNCTION, ONE QUADRATURE POINT

```
struct NonlinearDiffusion
{
    MFEM_HOST_DEVICE inline
    void operator()(const real_t &u,
                   const tensor<real_t, dim> &dudxi,
                   const tensor<real_t, dim, dim> &J,
                   const real_t &w,
                   tensor<real_t, dim> &dvdxi) const
    {
        const auto invJ = inv(J);
        const auto dudx = dudxi * invJ;
        const auto kappa = 1.0_r + u * u;
        dvdxi = kappa * dudx * transpose(invJ) * det(J) * w;
    }
};
```

■ κ coefficient

■ ∇u trial gradient

■ ∇v test function

■ $\det(J) w$ the measure

■ D the q-function

! **Inputs** are received in the **reference configuration**, and the quadrature weight and the geometric factors are **not** applied. Request **Weight** and geometric factors as an input and apply the **reference** → **physical** transformation in the q-function.

Anatomy of AddDomainIntegrator

```
template <typename backend_t = LocalQFBackend, ...>
void AddDomainIntegrator(
    qfunc_t          &qfunc,                // 1
    input_t          inputs,                // 2   Inputs<...> tuple
    output_t         outputs,               // 2   Outputs<...> tuple
    const IntegrationRule &integration_rule, // 3
    const Array<int>    &domain_attributes,  // 4
    derivative_ids_t    derivative_ids      = Derivatives<> {},           // 5
    second_derivative_ids_t second_derivative_ids = SecondDerivatives<Pairs::None> {});
```

- the **q-function** functor
- two tuples of **field operators**, one per **FieldDescriptor**
- an **IntegrationRule**, and the **attributes**
- (optional) which **derivatives** to generate

! The backend argument fixes how many quadrature points the q-function sees per call, and defaults to **LocalQFBackend**.



Using the operator

Apply it, ask for its Jacobian, and assemble a matrix if needed.

Applying the operator

```
MultiVector X{u_tdofs, mesh_nodes_tdofs}; // one block per input field
MultiVector Y{y_tdofs}; // one block per output field

dop.Mult(X, Y);
```

- One block per field, in **FieldDescriptor** order
- **SetMultLevel(level)** defines at what **level** of the decomposition the operator works on.



Requesting and using jacobians

GetDerivative hands back an **operator**, representing **derivative** for given **FieldID** **U**, **linearized around** state **X**

How much of it is precomputed is your choice, similar to **AssemblyLevel** for a **BilinearForm** (see Kernel registration)

MF

matrix-free action

DerivativeKernels::MF

```
auto dJ = dop.GetDerivative(U, X);
dJ->Mult(DU, DR);           // dr = J du
```

Recomputed on-the-fly at every apply.
No cache, no extra memory.

PA

cached setup, then apply

DerivativeKernels::PA

```
auto dJ = dop.GetDerivative(
    U, X, /*use_cached_setup=*/ true);
dJ->Mult(DU, DR);
dJ->MultTranspose(DR, DU);
```

Setup once, **cheap apply**. Compromise between memory/speed on repeated applications.

Assembled

a real matrix

DerivativeKernels::AllAssembly

```
dJ->Assemble(SparseMatrix *&A);
dJ->Assemble(HypreParMatrix *&A);

dJ->AssembleDiagonal(diag);
```

Triggers registration of callbacks required for assembly.
Nothing is assembled unless you ask.

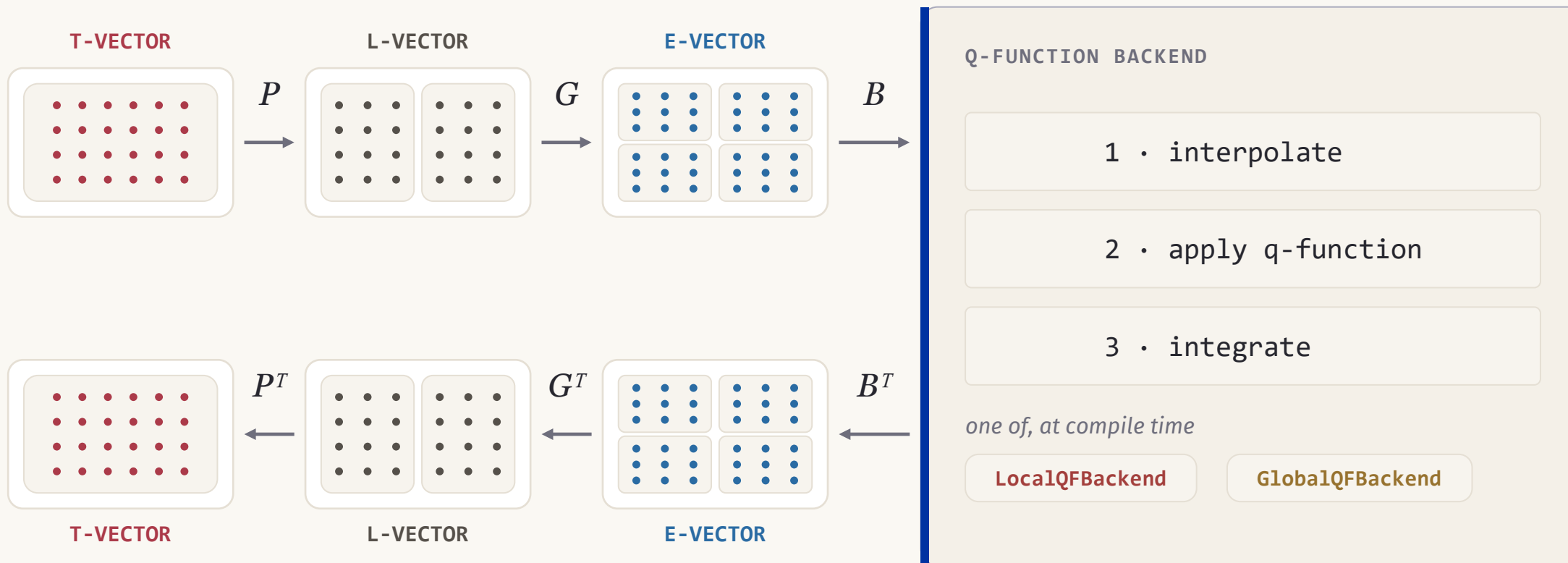


Local and global q-functions

DifferentiableOperator always owns $T \leftrightarrow L \leftrightarrow E$. The backend owns $E \rightarrow Q \rightarrow E$ and decides how many quadrature points your q-function sees per call.

Execution models in ∂ FEM: QFBackends

The way down to the E-vector and back up is **the same for both**. They differ only in the middle.



```
using IT = Inputs<Value<U>, Gradient<U>, Gradient<Coords>, Weight>;

dop.AddDomainIntegrator<LocalQFBackend>(qf_local, IT{}, OT{}, ir, attr, DT{});
dop.AddDomainIntegrator<GlobalQFBackend>(qf_global, IT{}, OT{}, ir, attr, DT{});
```

Which backend?

LocalQFBackend

- One quadrature point per call; **MFEM owns the loops**
- Generate **fused, tensor-product, GPU-friendly kernels**
- **Recommended** choice for most use cases

```
template <int DIM> struct local_mf_mass_qf
{
    inline MFEM_HOST_DEVICE
    void operator()(const dscalar_t &u,
                   const tensor<real_t, DIM, DIM> &J,
                   const real_t &w,
                   dscalar_t &v) const
    {
        v = u * w * det(J);
    }
};
```

GlobalQFBackend

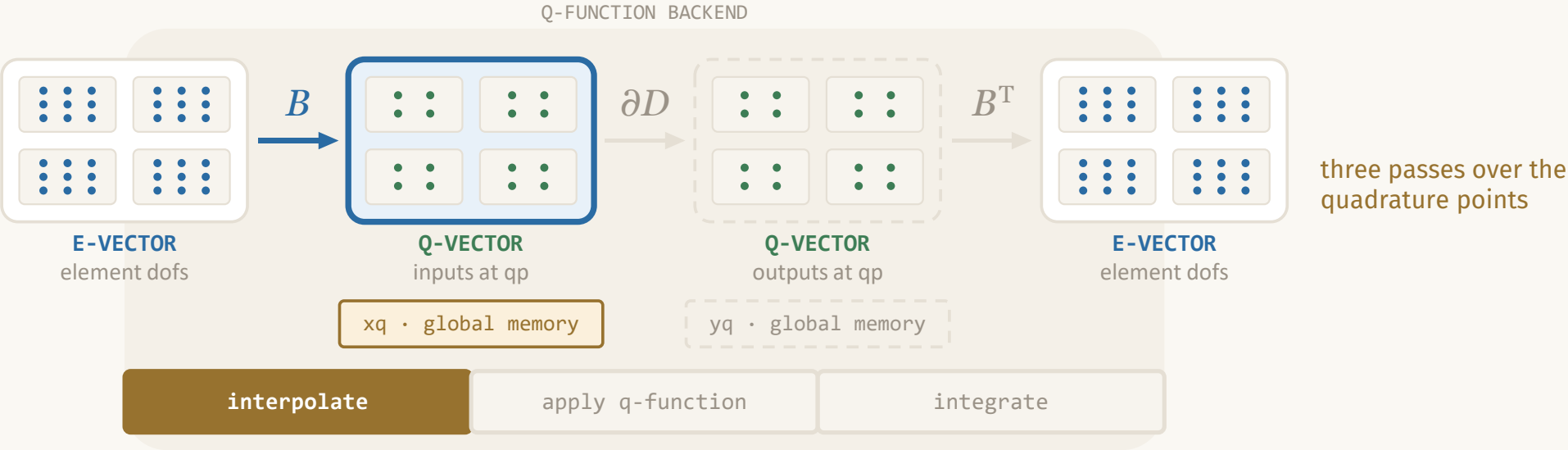
- **tensor_array** views spanning **all** quadrature points; **loop written inside the q-function**
- More complex use cases:
 - **Multiple kernels** called in the same q-function,
 - **External libraries** running on whole quadrature data

```
template <int DIM> struct global_mf_mass_qf
{
    inline MFEM_HOST_DEVICE
    void operator()(tensor_array<const dscalar_t> &u,
                   tensor_array<const real_t, DIM, DIM> &J,
                   tensor_array<const real_t> &w,
                   tensor_array<dscalar_t> &v) const
    {
        mfem::forall(u.size(), [=] MFEM_HOST_DEVICE (int q)
        {
            v(q) = (dscalar_t)(u(q)) * w(q) * det(J(q));
        });
    }
};
```

Stage 1 — interpolate

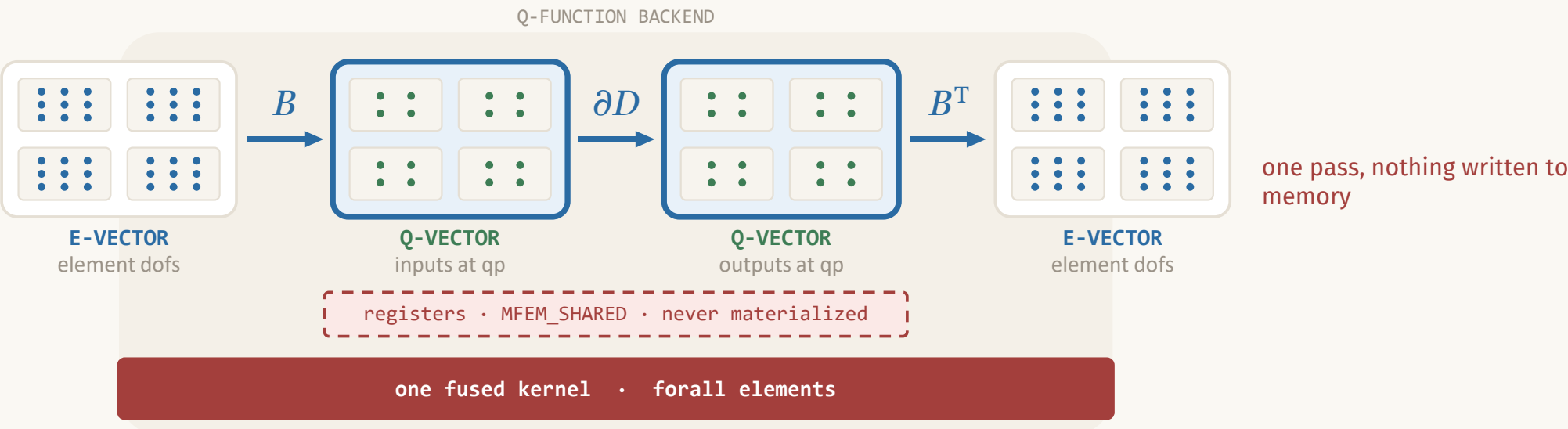
GlobalQFBackend

Q materialized in global memory



LocalQFBackend

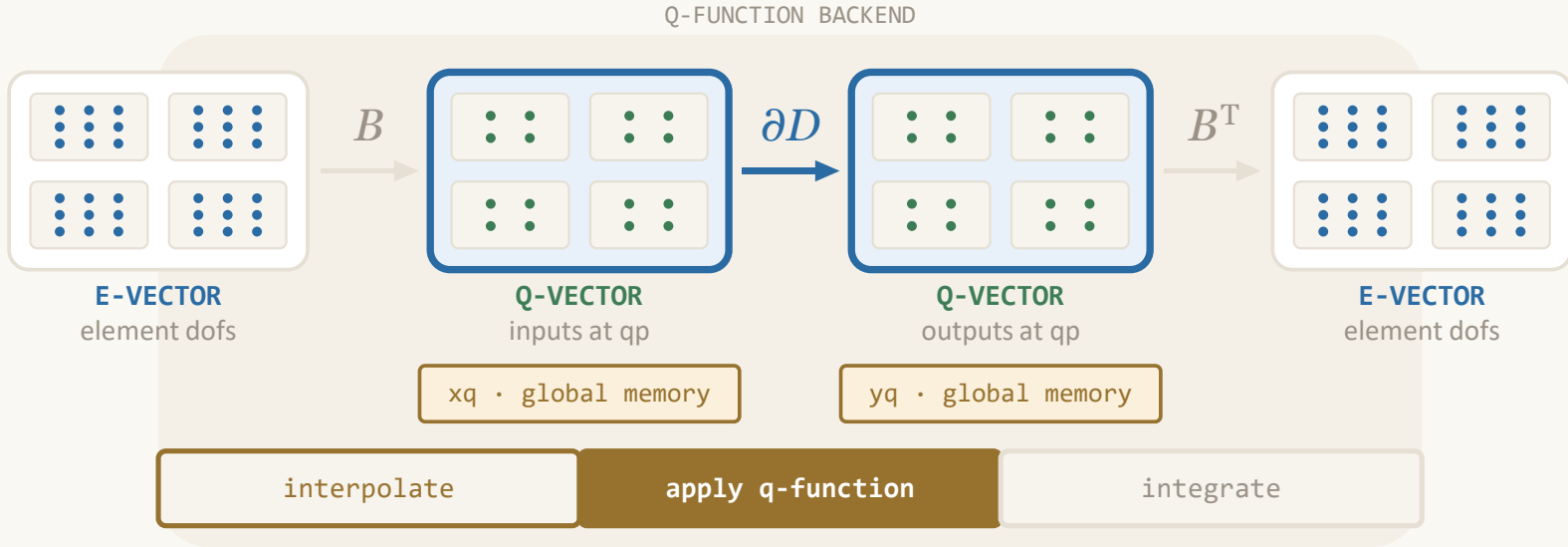
1 fused kernel · Q never leaves registers and shared



Stage 2 — the q-function

GlobalQFBackend

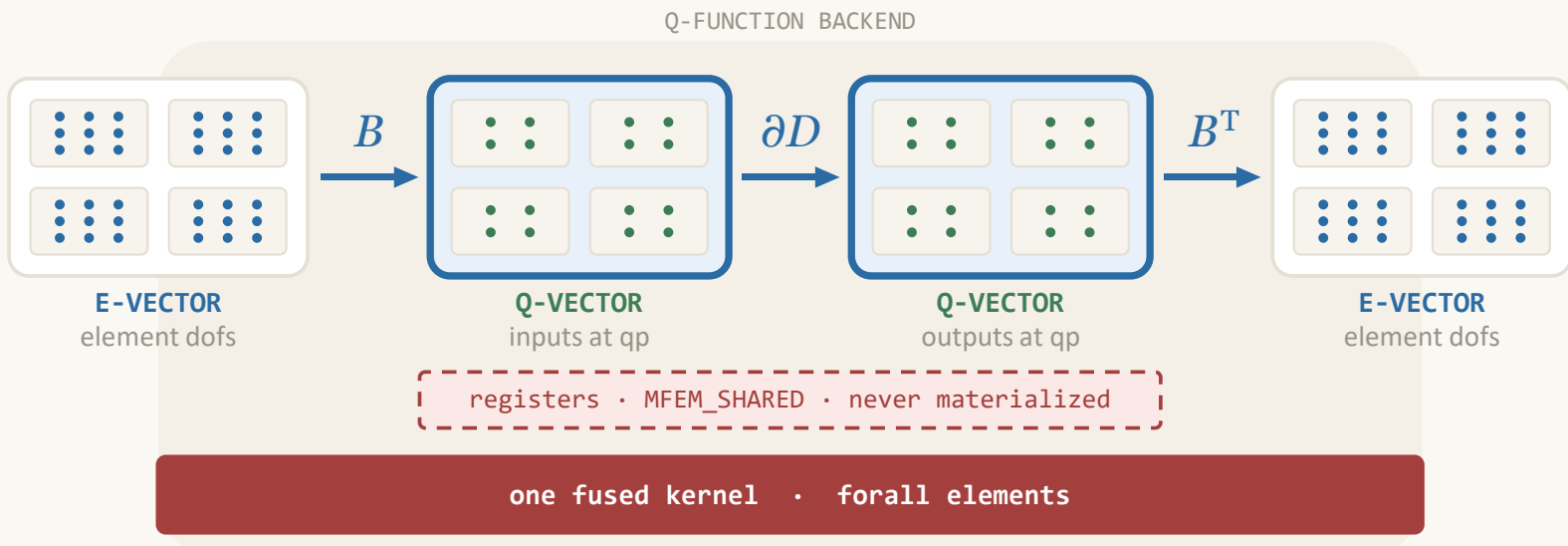
Q materialized in global memory



three passes over the quadrature points

LocalQFBackend

1 fused kernel · Q never leaves registers and shared

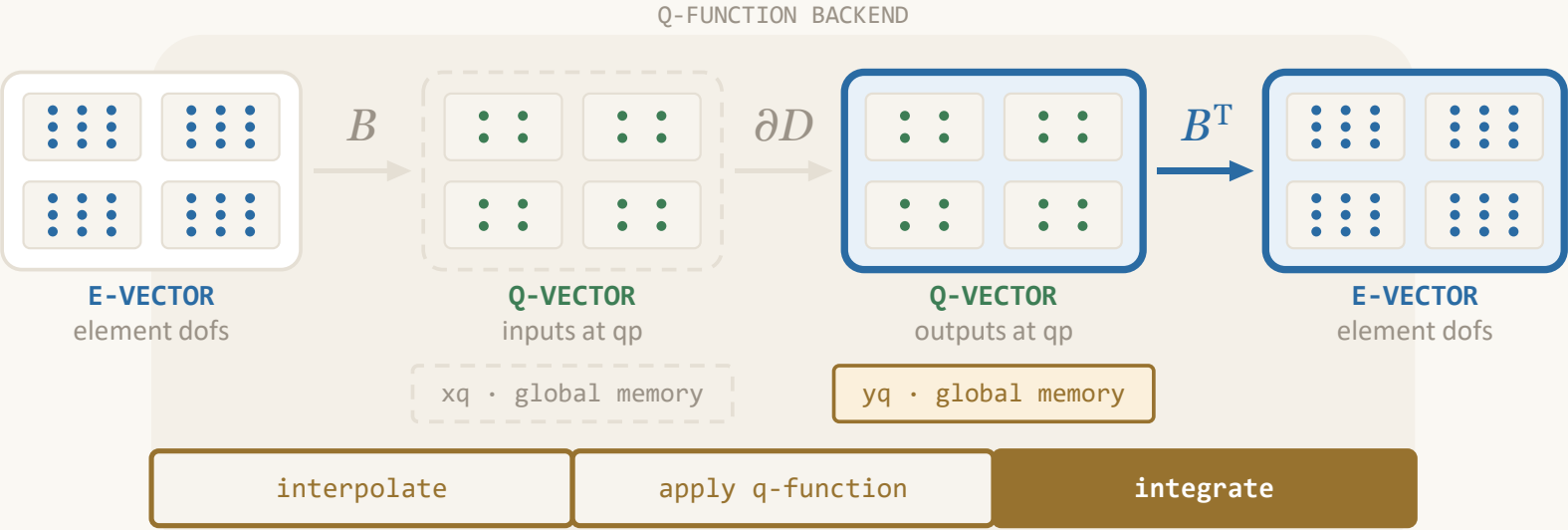


one pass, nothing written to memory

Stage 3 — integrate

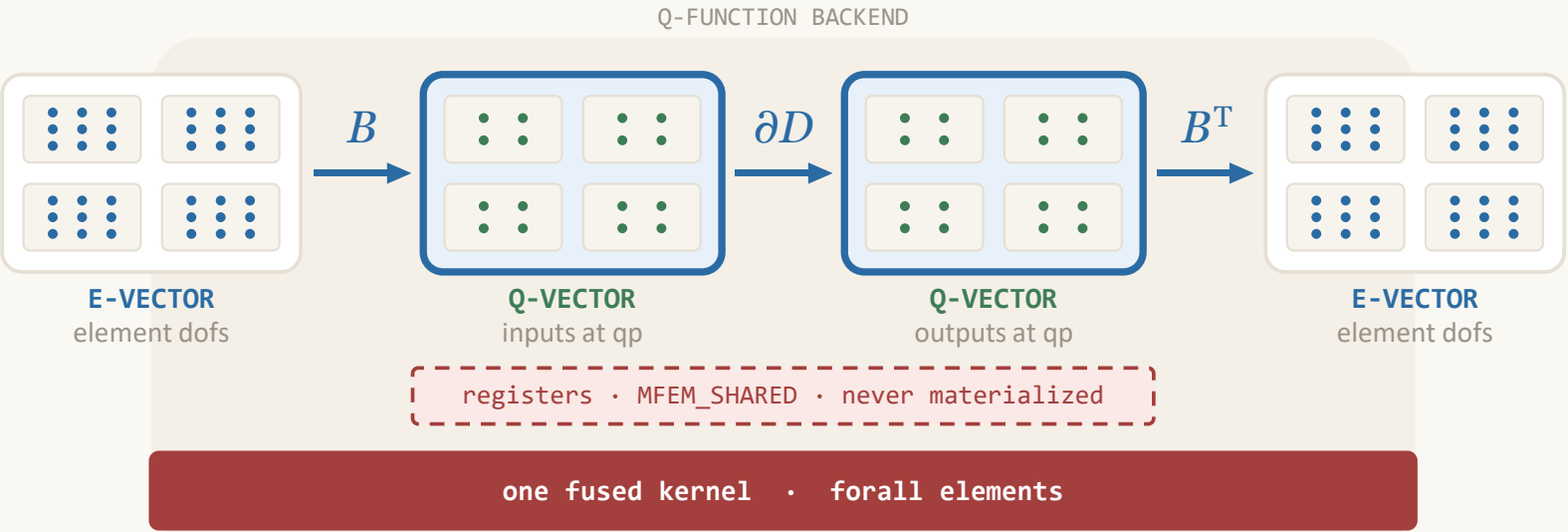
GlobalQFBackend

Q materialized in global memory



LocalQFBackend

1 fused kernel · Q never leaves registers and shared





Some additional features

Second derivatives, Optimize compile time, Examples and Performances

Second derivatives: functionals

When the physics is naturally a functional rather than a residual, writing the functional alone is enough.

$$F(u; \rho) = \int_{\Omega} \frac{1}{2} \rho |\nabla u|^2 dx$$

The **Output** lives on a quadrature space and **must be marked FunctionalValue**, which registers the integrator as a functional.

```
struct WeightedDirichletEnergy
{
    MFEM_HOST_DEVICE inline
    void operator()(const tensor<real_t, dim> &dudxi, // Gradient<U>
                   const real_t &rho, // Value<Rho>
                   const tensor<real_t, dim, dim> &J, // Gradient<Coords>
                   const real_t &w, // Weight
                   real_t &e) const // FunctionalValue<Energy>
    {
        const auto dudx = dudxi * inv(J);
        e = 0.5_r * rho * sqnorm(dudx) * det(J) * w; // already scaled by the measure
    }
};

std::vector<FieldDescriptor> outputs = {{Energy, &qspace_vec}};

F.AddDomainIntegrator<LocalQFBackend>(
    qf, Inputs <Gradient<U>, Value<Rho>, Gradient<Coords>, Weight> {},
        Outputs<FunctionalValue<Energy>> {},
    ir, all_domain_attr,
    Derivatives<U, Rho> {}, SecondDerivatives<Pairs::All> {});
```

Second derivatives: enabling Hessian blocks

Selector	Blocks
<code>Pairs::None</code>	none (default)
<code>Pairs::Diagonal</code>	only the diagonal blocks
<code>Pairs::All</code>	all blocks from the requested first derivatives
<code>DerivativePair<X, Y></code>	only $\partial/\partial Y (\nabla_X F)$

```
// Gradient: the residual  $r(u) = dF/du$ 
auto dFdu = F.GetDerivative(U);
MultiVector X{u, rho, mesh_nodes_tdoFs}, R{residual};
dFdu->Mult(X, R);

// Hessian block  $d/du (grad_u F)$ , linearised at X
F.GetSecondDerivative(U, X)->Mult(du, dr);

// Mixed block  $d/drho (grad_u F)$ 
F.GetSecondDerivative(U, Rho, X)->Mult(drho, dr);
```

! For a functional, `GetDerivative(id)` captures **no state** → you supply the full input at apply time through `Mult`.
`GetSecondDerivative(id, X)` does capture, like any other linearisation.

- Only functional integrators have them, and every pair's gradient id must also be a first derivative (checked at compile time).

Kernel selection: compile time and binary size

∂FEM is heavily templated, and Enzyme needs every instantiation at compile time.

The **number of registered kernels** inside a DifferentiableOperator is:

$$N = N_{dim} \times N_q \times (1 + N_k N_{\partial})$$

- **N_dim**: deduced from trailing extent of the first **Gradient** or **Hessian** argument. A q-function written against `tensor<scalar_t, 2, 2>` emits only 2D branch.
- **N_q**: quadrature tile sizes; fixed by the backend.
- **N_k**: **callbacks requested**, narrowed by a bit-mask.
- **N_∂**: requested derivatives; controlled by `Derivatives<...>`.

```
constexpr auto kernels =
    DerivativeKernels::Action | DerivativeKernels::Apply;

dop.AddDomainIntegrator<LocalQFBackend, kernels>(...);
```

Family	Expands to	For
None	nothing	only primal action
MF	Action	matrix-free action
PA	Apply ApplyTranspose	cached setup + apply
AllAssembly	AssembleMatrix AssembleDiagonal	matrices, diagonals
All	PA AllAssembly Action	the default

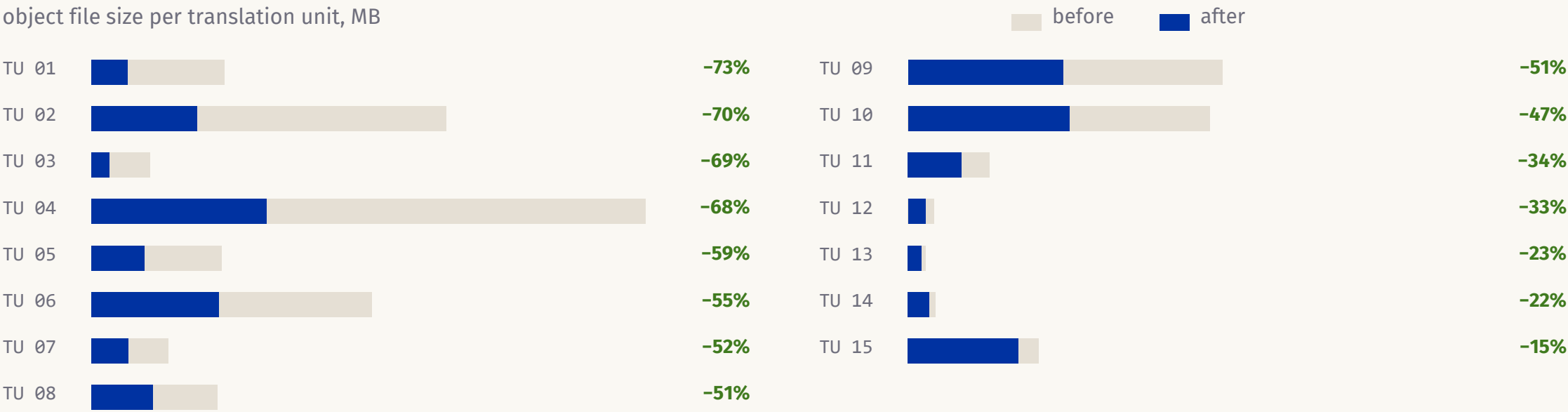
Setup is never requested explicitly; enabled by Apply, ApplyTranspose, AssembleMatrix and AssembleDiagonal.

Kernel selection: some results

The same fifteen translation units, built before and after optimization:

- **Deduced dimension** from the q-function signature
- **Only necessary kernels requested**

OBJECT CODE 137 → 60 MB -56%	PEAK RSS 2.13 → 0.98 GB -54%	COMPILE TIME 25 → 11 min -56%
--	--	---



Peak RSS is the worst single unit; compile time is the sum over all fifteen.

Examples: the ∂ FEM miniapps

Six miniapps, showcasing different features and ∂ FEM capabilities.



Minimal surface

$$\min \int_{\Omega} \sqrt{1 + |\nabla u|^2} d\Omega$$

AD against hand-coded derivatives and finite differences, side by side.



Hyperelasticity

$$E(u) = \int_{\Omega} \psi(\nabla u) d\Omega$$

Second derivatives of an energy functional; quasistatic 3D beam.



Mesh fitting/optimization with TMOP

$$\min \sum_T \int_T \mu(J(x)) + \sum_{s \in S} w \sigma^2(x_s)$$

MFEM version available at [MFEM · TMOP](#)



Hyperbolic heat transfer

$$\tau \partial_t^2 T + \rho c \partial_t T = \nabla \cdot (k(T) \nabla T) + Q$$

Nonlinear differentiation inside a time-dependent solve.



Reaction-diffusion

$$-\nabla \cdot (k \nabla u) + \alpha u + \beta u^3 = f$$

Multiple outputs for a single field, fused into one kernel.



Non-Newtonian Navier-Stokes

WIP

$$\rho(\partial_t u + (u \cdot \nabla)u) = \nabla \cdot \sigma(u, p), \quad \nabla \cdot u = 0$$

Multiple fields: mixed Taylor-Hood spaces, block residual, one derivative block per input field.

Benchmarks: BP3

[CEED bake-off problem 3](#) (scalar Laplace), 4.17 Mdofs

		local	global	setup	apply	vs MFEM	setup
<i>MFEM</i>							
	$\{G^T D G u\}$				29.5 ms	1.00×	8.5 ms
<i>cached apply</i>							
local , primal	$\{G^T \mathbf{Ql}(G u)\}$				33.3 ms	1.13×	8.7 ms
local , derivative	$\{G^T \mathbf{Ql}'(G u) G du\}$				33.4 ms	1.13×	10.8 ms
global , primal	$\{G^T \{\mathbf{Qg}(G u)\}\}$				97.1 ms	3.29×	14.2 ms
global , derivative	$\{G^T \{\mathbf{Qg}'(G u) G du\}\}$				33.4 ms	1.13×	20.5 ms
<i>matrix free</i>							
local , primal	$\{G^T \mathbf{Ql}(G u, G x)\}$				41.5 ms	1.41×	—
local , derivative	$\{G^T \mathbf{Ql}'(G u, G x) G du\}$				47.4 ms	1.61×	—
global , primal	$\{G^T \{\mathbf{Qg}(\{G u\}, \{G x\})\}\}$				104 ms	3.53×	—
global , derivative	$\{G^T \{\mathbf{Qg}'(\{G u\}, \{G x\}) \{G du\}\}\}$				181 ms	6.14×	—

Ql the local q-function, **Qg** the global one, ' the AD derivative, { } one kernel call (one fused, five for the global matrix-free derivative).

What changed since last year

Now shipped

was work in progress in 2025

- element matrix assembly
- processor-local **SparseMatrix**
- global **HypreParMatrix**
- partial assembly, cached setup
- linearized transpose action

New since 2025

did not exist in that talk

- **LocalQFBackend** / **GlobalQFBackend**
- **MultiVector**
- compile-time **Derivatives<...>**
- energies, **SecondDerivatives<...>**
- kernel-selection bit-masks

What's next

You will see it at the next workshop

- $H(\text{div})$ and $H(\text{curl})$ in ∂FEM
- Extended examples/miniapps
- Maybe more...

Where to go from here

1 Read a miniapp

`dfem-minimal-surface`

minimal surface in 2D

And upcoming examples

AVAILABLE AT [master · dfem](#)

2 Build it with Enzyme

```
cmake .. \  
-DMFEM_USE_MPI=ON \  
-DMFEM_USE_ENZYME=ON \  
-DENZYME_DIR=/path/to/enzyme
```

- Enzyme is a **Clang plugin** — build with a matching **clang++**, and **CUDA_CXX=clang++** for CUDA.
- **-d cuda** / **-d hip** at run time; the AD Jacobian runs on the GPU too.

3 Explore!

- Integrate **differentiability** in your code



CASC

Center for Applied
Scientific Computing



Lawrence Livermore National Laboratory

Disclaimer This document was prepared as an account of work sponsored by an agency of the United States government. Neither the United States government nor Lawrence Livermore National Security, LLC, nor any of their employees makes any warranty, expressed or implied, or assumes any legal liability or responsibility for the accuracy, completeness, or usefulness of any information, apparatus, product, or process disclosed, or represents that its use would not infringe privately owned rights. Reference herein to any specific commercial product, process, or service by trade name, trademark, manufacturer, or otherwise does not necessarily constitute or imply its endorsement, recommendation, or favoring by the United States government or Lawrence Livermore National Security, LLC. The views and opinions of authors expressed herein do not necessarily state or reflect those of the United States government or Lawrence Livermore National Security, LLC, and shall not be used for advertising or product endorsement purposes